

Documentación del Proyecto spaCEinvaders

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Curso: Paradigmas de Programación

Proyecto: spaCEinvaders

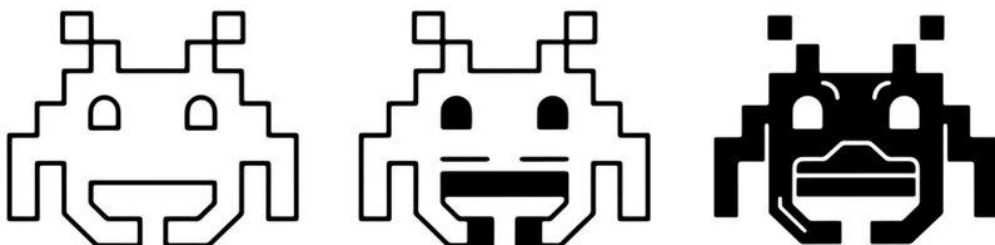
Integrantes:

- Andrés Blanco Coto 2022108841
- Gabriel Bolaños Barboza 2022327511
- Fabiana Moreno Castrillo 2023153700

Profesor:

Luis Barboza Artavia

11 de junio de 2026



1. Descripción de las funciones implementadas

El proyecto spaCEinvaders implementa un videojuego basado en Space Invaders utilizando una arquitectura cliente-servidor. El servidor fue desarrollado en Java y contiene la lógica principal de la partida, mientras que el cliente fue desarrollado en C y se encarga de mostrar la interfaz gráfica, capturar entradas del jugador y comunicarse con el servidor.

A continuación, se describen las principales funciones implementadas en el proyecto.

Servidor Java

El servidor Java es el componente central del sistema. Su función principal es mantener el estado real del juego y procesar todas las acciones realizadas durante la partida. Esto significa que los clientes no deciden directamente el resultado de las acciones, sino que envían solicitudes al servidor y este actualiza la partida.

Entre las funciones implementadas en el servidor se encuentran:

Gestión de jugadores

El servidor permite la conexión de al menos dos clientes jugadores. Cada jugador recibe un identificador único dentro de la partida. A partir de ese identificador, el servidor puede diferenciar las acciones de cada jugador, actualizar su posición, controlar sus vidas y almacenar su puntaje.

Cada jugador posee su propio cañón, su propio puntaje y su propia cantidad de vidas. Sin embargo, ambos jugadores participan dentro de la misma partida, por lo que observan los mismos extraterrestres, bunkers, proyectiles enemigos y ovni.

Gestión del cliente espectador

Además de los clientes jugadores, el servidor permite la conexión de un cliente espectador. Este cliente recibe el estado actualizado del juego, pero no puede enviar acciones de movimiento ni disparo. Su función es únicamente observar lo que ocurre en la partida.

Esto permite cumplir con el requerimiento de que un espectador pueda unirse a una partida existente y visualizar lo que sucede sin alterar el desarrollo del juego.

Movimiento de jugadores

El servidor recibe comandos enviados desde el cliente jugador para mover el cañón hacia la izquierda o hacia la derecha. Cuando el servidor recibe una acción de movimiento, verifica el jugador correspondiente y actualiza su posición dentro de los límites del área de juego.

El movimiento puede originarse desde el teclado del cliente gráfico o desde el control físico con ESP32. En ambos casos, el cliente convierte la entrada del usuario en una acción que se envía al servidor.

Disparo del jugador

El servidor permite que cada jugador realice disparos. Cuando un cliente envía la acción de disparo, el servidor genera un proyectil asociado al jugador correspondiente. Este proyectil se desplaza dentro del área de juego y puede impactar extraterrestres, bunkers u ovnis.

El servidor es el encargado de validar si el disparo impacta una entidad. Por esta razón, la lógica de colisiones no depende del cliente gráfico, sino del estado controlado por el servidor.

Gestión de extraterrestres

El servidor administra la lista de extraterrestres activos en la partida. Cada extraterrestre posee una posición, un estado activo y un valor de puntaje. Los extraterrestres pueden ser creados al inicio de la partida o mediante comandos administrativos ingresados en la consola del servidor.

Cuando un extraterrestre es impactado por un disparo del jugador, el servidor lo marca como eliminado y suma el puntaje correspondiente al jugador que realizó el disparo.

Los valores de puntaje utilizados siguen la lógica del juego original: diferentes tipos de extraterrestres pueden otorgar diferentes cantidades de puntos. En el proyecto se manejan aliens con valores como 10, 20 y 40 puntos, además de la posibilidad de crear extraterrestres con un valor específico mediante el comando administrativo Crear (X,Y,Pts).

Sistema de puntaje

El puntaje se administra desde el servidor. La partida es compartida por los jugadores conectados, por lo que ambos observan los mismos extraterrestres, bunkers, proyectiles enemigos y ovni. Sin embargo, cada jugador conserva su propio puntaje y sus propias vidas. Cuando un proyectil de un jugador impacta a un extraterrestre, el servidor identifica al dueño del disparo y suma los puntos correspondientes únicamente al jugador que realizó el impacto. De esta forma, el estado general de la partida es común, pero el marcador de cada jugador se mantiene de forma individual.

Además, el ovni puede otorgar un puntaje especial. Este puntaje se define al momento de crear el ovni desde la consola administrativa. Por ejemplo, el comando OVNI I-D 1500 crea un ovni que se desplaza de izquierda a derecha y puede otorgar 1500 puntos si es derribado.

De esta manera, el puntaje no se calcula en el cliente, sino que se mantiene centralizado en el servidor para evitar inconsistencias entre jugadores.

Sistema de vidas

Cada jugador inicia con 3 vidas. Cuando un jugador es impactado por un proyectil enemigo, el servidor reduce su cantidad de vidas en uno. Si las vidas del jugador llegan a cero, el jugador queda en estado de derrota.

También se implementó la regla de victoria por ronda. Cuando los jugadores eliminan todos los extraterrestres activos, el servidor reinicia la ronda, aumenta la velocidad de

desplazamiento de los enemigos y otorga una vida adicional al jugador. Esta regla permite que el juego continúe con una dificultad mayor.

Movimiento y velocidad de extraterrestres

El servidor controla el movimiento de los extraterrestres. Los enemigos se desplazan dentro del área de juego y su velocidad puede variar durante la ejecución.

La velocidad puede aumentar cuando se completa una ronda, es decir, cuando todos los extraterrestres son eliminados. También puede modificarse manualmente desde la consola administrativa mediante el comando Velocidad. Por ejemplo, Velocidad 180 modifica la velocidad de desplazamiento de los enemigos.

Gestión de bunkers

El juego incluye bunkers que funcionan como escudos de protección para los jugadores. Los bunkers pueden recibir daño por disparos y su resistencia se reduce gradualmente.

El servidor mantiene el estado de cada bunker y comunica ese estado a los clientes para que sea representado gráficamente. Además, el administrador puede modificar el porcentaje de resistencia de los bunkers mediante comandos como:

Bunkers 70%

Bunkers 40%

Bunkers 0%

Esto permite probar en tiempo real diferentes estados de protección dentro de la partida.

Gestión del ovni

El servidor permite crear un ovni durante la partida. Este ovni puede desplazarse de izquierda a derecha o de derecha a izquierda, dependiendo del comando utilizado por el administrador.

Los comandos utilizados son:

OVNI I-D 1500

OVNI D-I 1500

El primer comando crea un ovni que se mueve de izquierda a derecha, mientras que el segundo crea un ovni que se mueve de derecha a izquierda. El valor numérico indica el puntaje asociado al ovni.

Comandos administrativos

El servidor implementa una consola administrativa que permite modificar elementos del juego en tiempo de ejecución. Esta funcionalidad permite probar diferentes escenarios sin detener la partida.

Los principales comandos administrativos son:

Crear (X,Y,Pts)
OVNI I-D 1500
OVNI D-I 1500
Velocidad 100
Bunkers 70%
Bunkers 40%
Bunkers 0%
ayuda
salir

Estos comandos permiten crear extraterrestres, generar ovnis, modificar la velocidad, cambiar el estado de los bunkers, consultar ayuda y cerrar el servidor correctamente.

Comunicación por sockets

La comunicación entre el servidor Java y los clientes C se realiza mediante sockets. Los clientes envían mensajes al servidor cuando el jugador se mueve, dispara o se desconecta. El servidor procesa esos mensajes y luego envía el estado actualizado del juego a todos los clientes conectados.

Este sistema permite que dos jugadores y un espectador se mantengan sincronizados en la misma partida.

Cliente gráfico en C

El cliente gráfico fue desarrollado en C utilizando la librería raylib. Su función principal es mostrar visualmente el estado del juego y permitir que el usuario interactúe con la partida.

Entre las funciones implementadas en el cliente se encuentran:

Conexión con el servidor

El cliente se conecta al servidor mediante sockets. Al conectarse, recibe un identificador si es jugador o se registra como espectador si se ejecuta en modo observador.

Después de conectarse, el cliente recibe continuamente mensajes con el estado actualizado del juego.

Interpretación del estado del juego

El cliente interpreta los mensajes enviados por el servidor y actualiza sus estructuras internas. Estas estructuras contienen la información necesaria para dibujar jugadores, extraterrestres, proyectiles, bunkers, ovni, puntajes, vidas y velocidad.

El cliente no calcula la lógica principal del juego. Su función es representar gráficamente lo que el servidor envía.

Renderizado gráfico

El cliente gráfico dibuja la pantalla del juego utilizando raylib. En la ventana se muestran los jugadores, aliens, proyectiles, bunkers, ovni, puntajes, vidas, velocidad y estado general de la partida.

También se muestra información visual que permite diferenciar al jugador local, los demás jugadores y el modo espectador.

Entrada por teclado

El cliente permite controlar al jugador mediante teclado. Los controles implementados son:

A: mover a la izquierda

D: mover a la derecha

F: disparar

Espacio: disparar

Q o ESC: salir

Cuando el usuario presiona una tecla de movimiento o disparo, el cliente envía la acción correspondiente al servidor.

Cliente espectador

El mismo cliente gráfico puede ejecutarse en modo espectador. En este modo, el cliente recibe el estado del juego y lo dibuja en pantalla, pero no envía comandos de movimiento ni disparo.

Esto permite observar la partida sin modificarla.

Soporte para control físico ESP32

El cliente gráfico también incluye soporte para un control físico basado en ESP32. La ESP32 se comunica con el cliente por UART y envía comandos simples:

L: izquierda

R: derecha

F: disparar

El cliente interpreta estos comandos y los transforma en acciones del jugador. Luego, estas acciones son enviadas al servidor por sockets.

La ESP32 no se comunica directamente con el servidor. Su función es actuar como dispositivo de entrada para el cliente gráfico.

Cierre del cliente

El cliente puede cerrarse mediante Q o ESC. Al cerrarse, libera los recursos utilizados, cierra la ventana gráfica y finaliza la conexión con el servidor.

Resumen de funciones implementadas

En resumen, el proyecto implementa las siguientes funcionalidades principales:

- Servidor Java con lógica central del juego.
- Cliente gráfico en C.
- Comunicación cliente-servidor mediante sockets.
- Soporte para dos clientes jugadores.
- Soporte para cliente espectador.
- Movimiento y disparo de jugadores.
- Gestión de vidas y puntaje.
- Gestión de extraterrestres.
- Gestión de proyectiles.
- Gestión de bunkers.
- Gestión de ovni.
- Comandos administrativos en tiempo de ejecución.
- Reinicio de ronda con aumento de velocidad.
- Soporte para control físico ESP32 por UART.
- Uso de patrones de diseño en Java.
- Uso de estructuras en C para representar el estado del juego.

2. Descripción y ejemplificación de las estructuras de datos desarrolladas

En el proyecto spaCEinvaders se utilizaron diferentes estructuras de datos para organizar la información del juego, tanto en el servidor Java como en el cliente C. Estas estructuras permiten representar jugadores, extraterrestres, proyectiles, bunkers, ovni, mensajes de red y el estado general de la partida.

La utilización de estas estructuras fue necesaria porque el juego maneja múltiples elementos al mismo tiempo. Por ejemplo, pueden existir varios jugadores conectados, varios extraterrestres activos, múltiples proyectiles en pantalla y varios bunkers con diferentes niveles de resistencia.

Estructuras de datos en el servidor Java

El servidor Java es el encargado de mantener toda la lógica principal del juego. Por esta razón, en esta parte se utilizaron clases para representar las entidades de la partida y listas para manejar colecciones de objetos.

Clase GameState

La clase GameState representa el estado general de la partida. Esta estructura es una de las más importantes del servidor, ya que contiene y administra la información necesaria para controlar el juego.

Dentro de GameState se maneja información como:

- Jugadores conectados.

- Extraterrestres activos.
- proyectiles.
- Bunkers.
- Ovni.
- Velocidad del juego.
- Estado general de la partida.
- Reglas de colisión.
- Reinicio de ronda.
- Actualización de vidas y puntajes.

Esta clase permite centralizar la lógica del juego. Por ejemplo, cuando un jugador dispara, el servidor registra el proyectil dentro del estado del juego. Luego, durante la actualización de la partida, se revisa si ese proyectil impacta contra un extraterrestre, bunker u ovni.

Ejemplo de uso dentro del juego:

Jugador dispara

...

GameState registra el proyectil

...

El proyectil se mueve

...

GameState revisa colisiones

...

Si impacta un alien, el alien se elimina y se suma puntaje

Clase Player

La clase Player representa a cada jugador conectado a la partida. Cada jugador tiene información individual, aunque participe dentro de la misma partida compartida.

Esta estructura almacena datos como:

- Identificador del jugador.
- Nombre o referencia del jugador.
- Posición del cañón.
- Vidas disponibles.
- Puntaje acumulado.
- Estado dentro de la partida.

Esta estructura permite que el servidor diferencie las acciones de cada jugador. Por ejemplo, si el jugador 1 dispara y elimina un alien, el puntaje se suma al jugador 1 y no al jugador 2.

Ejemplo:

Jugador 1 elimina un alien de 20 puntos

...

Servidor identifica que el disparo pertenece al jugador 1

...

Se suman 20 puntos al marcador del jugador 1

...

El jugador 2 mantiene su propio puntaje

La partida es compartida, pero cada jugador conserva sus datos individuales, como vidas y puntaje.

Clase Alien

La clase Alien representa a los extraterrestres del juego. Cada alien posee una posición dentro del mundo, un valor de puntaje y un estado que indica si se encuentra activo o eliminado.

Esta estructura permite que el servidor controle a cada enemigo de forma individual. Cuando un proyectil impacta a un alien, el servidor puede identificar cuál alien fue alcanzado y eliminarlo de la partida.

Ejemplo:

Alien en posición X=300, Y=120, valor=20 puntos

...

Proyectil del jugador llega a esa zona

...

El servidor detecta colisión

...

El alien queda eliminado

...

Se suman 20 puntos al jugador correspondiente

Los aliens también pueden ser creados desde la consola administrativa del servidor mediante comandos como:

Crear (300,120,20)

Esto permite agregar un extraterrestre en una posición específica con un valor de puntaje determinado.

Clase Projectile

La clase Projectile representa los disparos que se encuentran activos en la partida. Estos proyectiles pueden pertenecer a un jugador o a los enemigos.

Esta estructura almacena información como:

- Posición del proyectil.
- Dirección de movimiento.

- Estado activo o inactivo.
- Dueño del disparo, cuando corresponde.

La información del dueño del disparo es importante porque permite saber a qué jugador se le debe sumar el puntaje cuando un proyectil elimina un extraterrestre.

Ejemplo:

Jugador 2 dispara

...

Se crea un proyectil asociado al jugador 2

...

El proyectil impacta un alien de 40 puntos

...

El servidor suma 40 puntos al jugador 2

También se utilizan proyectiles enemigos. Si un proyectil enemigo impacta a un jugador, el servidor reduce las vidas de ese jugador.

Clase Bunker

La clase Bunker representa los escudos de protección del juego. Cada bunker tiene una posición y un porcentaje de resistencia.

Los bunkers sirven como protección entre los jugadores y los disparos enemigos. Cuando un proyectil impacta a un bunker, su resistencia disminuye. Si la resistencia llega a cero, el bunker deja de funcionar como protección.

Ejemplo:

Bunker con 70% de resistencia

...

Recibe un impacto

...

Su resistencia disminuye

...

El cliente gráfico lo dibuja más dañado

El estado de los bunkers también puede modificarse desde la consola administrativa del servidor mediante comandos como:

Bunkers 70%

Bunkers 40%

Bunkers 0%

Esto permite probar distintos niveles de daño durante la ejecución de la partida.

Clase UFO

La clase UFO representa el ovni que puede aparecer durante el juego. Esta estructura almacena información como:

- Estado activo o inactivo.
- Posición.
- Dirección de movimiento.
- Puntaje asociado.

El ovni puede ser creado desde la consola administrativa del servidor. Por ejemplo:

OVNI I-D 1500

Este comando crea un ovni que se mueve de izquierda a derecha y que otorga 1500 puntos si es derribado.

También se puede crear en sentido contrario:

OVNI D-I 1500

La estructura del ovni permite controlar su aparición, movimiento, puntaje y eliminación.

Listas utilizadas en Java

En el servidor se utilizan listas para manejar grupos de objetos. Estas listas permiten almacenar varios elementos del mismo tipo y recorrerlos durante la actualización del juego.

Algunas listas importantes son:

Lista de jugadores
Lista de extraterrestres
Lista de proyectiles
Lista de bunkers
Lista de observadores

La lista de jugadores permite manejar a los clientes conectados. La lista de extraterrestres permite recorrer todos los aliens activos para moverlos y verificar colisiones. La lista de proyectiles permite actualizar los disparos en pantalla. La lista de bunkers permite revisar daños y actualizar sus porcentajes. La lista de observadores permite notificar a los clientes cuando cambia el estado del juego.

Ejemplo de uso de la lista de extraterrestres:

El servidor recorre la lista de aliens

...

Verifica si cada alien sigue activo

...

Actualiza su posición

...

Revisa si algún proyectil lo impactó

...

Si fue impactado, lo marca como eliminado

Ejemplo de uso de la lista de observadores:

GameServer actualiza el estado del juego

...

Llama a notifyObservers

...

Cada ClientHandler recibe el nuevo estado

...

Los clientes gráficos actualizan la pantalla

Esta lista está relacionada con el patrón Observer, ya que permite que el servidor notifique a todos los clientes conectados sin depender directamente de cada uno de ellos.

Estructuras de datos en el cliente C

En el cliente C se utilizaron structs para representar la información recibida desde el servidor. Estas estructuras permiten guardar temporalmente el estado de la partida para dibujarlo en la ventana gráfica.

El cliente no mantiene la lógica principal del juego. Su función es recibir el estado actualizado, interpretarlo y representarlo gráficamente.

Struct GameState

En el cliente C, el struct GameState representa el estado recibido desde el servidor. Esta estructura contiene la información necesaria para dibujar la partida en pantalla.

Incluye datos como:

- Estado general del juego.
- Jugadores.
- Extraterrestres.
- Proyectiles.
- Bunkers.
- Ovni.
- Velocidad.
- Cantidad de aliens activos.
- Información de vidas y puntajes.

Ejemplo de uso:

Cliente recibe mensaje STATE desde el servidor

...

Interpreta los datos recibidos

...

Llena el struct GameState

...

Dibuja jugadores, aliens, bunkers, proyectiles y ovni

Esta estructura permite que el cliente gráfico trabaje con una copia local del estado enviado por el servidor.

Struct Player

El struct Player en C permite representar gráficamente a cada jugador. Contiene la información necesaria para dibujar su cañón, mostrar sus vidas y visualizar su puntaje.

Ejemplo:

Servidor envía posición del jugador 1

...

Cliente guarda esa posición en un struct Player

...

Cliente dibuja el cañón del jugador 1 en pantalla

También permite diferenciar al jugador local de otros jugadores conectados.

Struct Alien

El struct Alien permite representar a los extraterrestres en el cliente gráfico. La información recibida desde el servidor se almacena en estas estructuras para saber dónde dibujar cada alien y si se encuentra activo.

Ejemplo:

Alien activo en X=400, Y=100

...

Cliente lo recibe desde el servidor

...

Cliente lo dibuja en la zona correspondiente de la pantalla

Si el servidor indica que un alien fue eliminado, el cliente deja de dibujarlo.

Struct Projectile

El struct Projectile representa los disparos activos en pantalla. El cliente utiliza esta estructura para dibujar los proyectiles enviados por el servidor.

Ejemplo:

Servidor indica que hay un proyectil en X=250, Y=300

...

Cliente guarda esa información

...

Cliente dibuja el disparo en pantalla

El movimiento real y la validación de impactos se realizan en el servidor, el cliente únicamente muestra el resultado actualizado.

Struct Bunker

El struct Bunker permite representar los escudos de protección. El cliente utiliza la posición y resistencia del bunker para dibujarlo con el nivel de daño correspondiente.

Ejemplo:

Bunker al 40%

...

Cliente recibe ese porcentaje

...

Cliente lo dibuja visualmente más dañado

Esto permite que los jugadores identifiquen el estado de protección disponible durante la partida.

Struct UFO

El struct UFO representa el ovni en el cliente gráfico. El cliente utiliza esta estructura para saber si el ovni está activo, en qué posición se encuentra y hacia dónde se mueve.

Ejemplo:

Servidor crea un OVNI I-D 1500

...

Cliente recibe que el ovni está activo

...

Cliente lo dibuja moviéndose de izquierda a derecha

Arreglos en el cliente C

En el cliente C se utilizan arreglos para almacenar múltiples entidades dentro del estado del juego. Por ejemplo, se utilizan arreglos para jugadores, aliens, proyectiles y bunkers.

Este enfoque permite recorrer los elementos fácilmente durante el renderizado.

Ejemplo:

El cliente recorre el arreglo de aliens

...

Verifica cuáles están activos

...

Dibuja cada alien activo en pantalla

Los arreglos son adecuados para el cliente porque permiten manejar una cantidad máxima definida de elementos y simplifican la representación gráfica.

Constantes en archivo separado

El cliente C también utiliza constantes definidas en un archivo separado. Estas constantes permiten controlar valores importantes del juego y de la interfaz, como dimensiones del mundo, tamaños máximos y configuraciones generales.

Esto evita escribir valores fijos directamente en varias partes del código y facilita realizar modificaciones futuras.

Ejemplo:

Si se desea cambiar el ancho del mundo de juego, se modifica la constante correspondiente, sin alterar directamente la lógica de dibujo o comunicación.

Estructuras relacionadas con UART

Para integrar el control físico con ESP32, se agregó una estructura de comunicación serial en el cliente C. El cliente abre un puerto COM y lee comandos enviados por UART.

La ESP32 envía comandos simples:

L: izquierda

R: derecha

F: disparar

El cliente interpreta estos comandos y los convierte en acciones del jugador. Luego, esas acciones se envían al servidor mediante sockets.

Ejemplo:

ESP32 envía L por UART

...

Cliente C lee el comando

...

Cliente interpreta L como MOVE_LEFT

...
Cliente envía MOVE_LEFT al servidor
...
Servidor mueve el jugador a la izquierda

Importancia de las estructuras de datos

Las estructuras de datos utilizadas permiten que el proyecto mantenga una organización clara entre la lógica del servidor y la visualización del cliente.

En Java, las clases y listas permiten manejar la lógica de la partida de forma orientada a objetos. En C, los structs y arreglos permiten representar el estado del juego de forma simple y eficiente para mostrarlo gráficamente.

Gracias a estas estructuras, el proyecto puede manejar varios jugadores, múltiples aliens, proyectiles, bunkers, ovni, espectador y control físico, manteniendo una comunicación ordenada entre el servidor y los clientes.

3. Problemas sin solución

Al finalizar el desarrollo del proyecto spaCEinvaders, no quedaron problemas críticos sin solución dentro del código principal del juego. El servidor Java compila y ejecuta correctamente, el cliente gráfico en C se conecta al servidor, se pueden abrir dos clientes jugadores, se puede abrir un cliente espectador y la comunicación por sockets funciona correctamente.

También se verificó que la lógica principal del juego funciona desde el servidor. El sistema permite mover jugadores, disparar, eliminar extraterrestres, sumar puntaje, modificar vidas, mostrar bunkers, crear ovnis, cambiar velocidad y utilizar comandos administrativos desde la consola del servidor. Además, se comprobó que al eliminar todos los extraterrestres se reinicia la ronda, se incrementa la velocidad del juego y se otorga una vida adicional al jugador.

El soporte para control físico mediante ESP32 también fue integrado en el cliente gráfico. El cliente puede leer comandos por UART y convertirlos en acciones del jugador. Los comandos esperados son:

L:	mover	a	la	izquierda
R:	mover	a	la	derecha
F:	disparar			

Estos comandos son enviados por el control físico y luego el cliente los transforma en acciones que se comunican al servidor por sockets.

El único aspecto que hay que considerar durante la revisión es que el uso del control físico depende de que la computadora utilizada tenga correctamente configurado el entorno necesario, incluyendo el puerto COM de la ESP32, el compilador GCC y la librería raylib para

el cliente gráfico. Este punto no es un problema sin solución del código, sino una condición de instalación y prueba del equipo físico.

Por lo que, no quedaron funcionalidades principales sin resolver. Las pruebas pendientes correspondientes a las pruebas con el control fueron resueltas de igual manera.

5. Plan de actividades

Actividad	Descripción	Tiempo estimado	Responsable	Fecha de entrega
Diseño de la arquitectura cliente-servidor	Definición de la estructura general del proyecto, protocolos de comunicación y distribución de responsabilidades entre servidor y clientes.	10 horas	Andrés Blanco	18/05/2026
Desarrollo del servidor Java	Implementación de la lógica principal del juego, gestión de jugadores, aliens, proyectiles, bunkers, ovni y comandos administrativos.	35 horas	Fabiana Moreno	30/05/2026
Implementación de patrones de diseño	Integración de patrones orientados a objetos para cumplir los requisitos del proyecto y mejorar la organización del código.	12 horas	Andrés Blanco	01/06/2026
Desarrollo del cliente gráfico en C	Implementación de la interfaz gráfica utilizando raylib, renderizado de entidades y comunicación con el servidor.	25 horas	Fabiana moreno	02/06/2026
Implementación de comunicación por sockets	Desarrollo de la comunicación entre cliente y servidor para intercambio de acciones y estados de juego.	12 horas	Andres blanco	03/06/2026

confección del circuito y case del control	Diseño el circuito, implementarlo en placa perforada y confeccionar el modelo del case para el control.	8 horas	Gabriel Bolaños	06/06/2026
Integración del control físico ESP32	Lectura de comandos por UART e integración con el cliente gráfico para controlar el jugador mediante hardware externo.	15 horas	Gabriel Bolaños	06/06/2026
Diseño y pruebas de jugabilidad	Verificación de movimiento, disparos, colisiones, sistema de vidas, puntajes y reinicio de rondas.	12 horas	Fabiana Moreno	07/06/2026
Pruebas de integración	Verificación de funcionamiento conjunto entre servidor, clientes jugadores, espectador y control físico.	10 horas	Todos los integrantes	09/06/2026
Elaboración de documentación	Redacción de la documentación técnica, descripción de estructuras de datos, problemas encontrados y manuales.	12 horas	Todos los integrantes	10/06/2026
Revisión final y preparación de defensa	Validación del funcionamiento completo del proyecto y preparación de la presentación final.	6 horas	Todos los integrantes	11/06/2026

Total estimado del proyecto: 157 horas de trabajo distribuidas entre los integrantes del grupo.

6. Problemas encontrados

Durante el desarrollo del proyecto spaCEinvaders se presentaron diferentes problemas técnicos relacionados con la configuración del entorno, la compilación del servidor y del cliente, la comunicación por sockets, la interfaz gráfica, las colisiones del juego, el control físico y la organización del repositorio. A continuación, se describen los principales problemas encontrados, los intentos de solución realizados, la solución final aplicada, las recomendaciones y las conclusiones obtenidas.

Problema 1: Configuración del entorno Java y Maven

Uno de los primeros problemas encontrados fue la configuración del entorno necesario para compilar y ejecutar el servidor Java. Aunque Java estaba instalado en la computadora, inicialmente no se contaba con una configuración completa para compilar el proyecto con Maven. Esto impedía generar correctamente el archivo ejecutable .jar del servidor.

Intentos de solución

Primero se intentó verificar la instalación de Java mediante comandos como:

```
java -version  
javac -version
```

Sin embargo, se observó que no bastaba con tener Java instalado, ya que también era necesario contar con el JDK completo y con Maven correctamente configurado en el sistema.

Solución encontrada

Se instaló y configuró un JDK compatible y Apache Maven. Posteriormente se verificó la instalación con:

```
mvn -v
```

Después de configurar correctamente Maven, el servidor pudo compilarse mediante:

```
mvn clean package
```

Este comando generó el archivo .jar necesario para ejecutar el servidor.

Recomendación

Se recomienda verificar desde el inicio que todos los integrantes tengan instalado Java JDK y Maven, y no únicamente Java Runtime. También se recomienda documentar la versión utilizada para evitar errores de compatibilidad durante la revisión.

Conclusión

Este problema permitió confirmar la importancia de configurar correctamente el entorno antes de comenzar las pruebas funcionales. Sin un entorno Java y Maven funcional, el servidor no puede compilarse y el proyecto no puede ejecutarse correctamente.

Bibliografía consultada

- Documentación oficial de Apache Maven.
- Documentación oficial de Java JDK.
- Material del curso relacionado con compilación y ejecución de proyectos Java.

Problema 2: Compilación del cliente gráfico en C con raylib

Otro problema importante fue la compilación del cliente gráfico en C. El cliente utiliza la librería raylib para mostrar la interfaz gráfica del juego. En algunas computadoras, el compilador GCC estaba instalado, pero no encontraba el archivo raylib.h, generando errores como:

```
fatal error: raylib.h: No such file or directory
```

Esto impedía compilar el cliente gráfico.

Intentos de solución

Primero se intentó compilar el cliente gráfico utilizando el script:

```
.\scripts\build-graphic-client.bat
```

Sin embargo, el script solo indicaba que no se había podido compilar, sin mostrar suficiente detalle. Por esta razón, se ejecutó manualmente el comando de compilación con gcc para identificar el error real.

También se verificó si el compilador estaba instalado con:

```
gcc --version  
where.exe gcc
```

Con esto se pudo identificar que el problema no era necesariamente la ausencia de GCC, sino que el GCC utilizado no tenía acceso a la instalación de raylib.

Solución encontrada

La solución fue instalar raylib en el entorno correcto de MSYS2 UCRT64 y asegurarse de que PowerShell utilizara el GCC correspondiente a ese entorno. Una vez configurado correctamente, el cliente gráfico pudo compilarse mediante el script del proyecto.

Recomendación

Se recomienda que la computadora utilizada para la defensa tenga previamente instalado MSYS2 UCRT64, GCC y raylib.

Conclusión

Este problema evidenció que no basta con tener un compilador de C instalado, sino que también deben estar correctamente configuradas las librerías externas utilizadas por el proyecto. En este caso, raylib era indispensable para compilar y ejecutar el cliente gráfico.

Bibliografía consultada

- Documentación oficial de raylib.
- Documentación de MSYS2.
- Material de apoyo sobre compilación de programas en C.

Problema 3: Cliente C bloqueado al recibir mensajes del servidor

Durante las primeras pruebas del cliente C, se presentó un problema relacionado con la recepción de mensajes del servidor. El cliente quedaba esperando datos del socket y esto afectaba la lectura de entradas del usuario. Como consecuencia, el jugador podía experimentar retrasos o dificultades para moverse y disparar correctamente.

Intentos de solución

Inicialmente se intentó mantener una lectura directa del socket en el ciclo principal del cliente. Sin embargo, este enfoque provocaba que el programa se quedara esperando mensajes del servidor, afectando la fluidez de la entrada por teclado.

También se realizaron pruebas para verificar si el problema venía del servidor, pero se concluyó que el servidor sí estaba enviando datos correctamente. El problema estaba en la forma en que el cliente esperaba esos datos.

Solución encontrada

Se implementó una revisión previa para saber si había datos disponibles en el socket antes de intentar leerlos. Para esto se agregó una función que permite consultar si el socket tiene información pendiente. De esta forma, el cliente solo intenta leer datos cuando realmente hay algo disponible.

Esto permitió que el ciclo principal del cliente continuara ejecutándose sin bloquear la lectura del teclado ni el renderizado gráfico.

Recomendación

Se recomienda evitar lecturas bloqueantes dentro del ciclo principal de un cliente gráfico o interactivo. En aplicaciones en tiempo real, como videojuegos, es importante que el ciclo principal pueda seguir ejecutándose aunque no haya datos nuevos disponibles desde el servidor.

Conclusión

Este problema permitió mejorar la estabilidad del cliente y separar mejor la lógica de entrada del usuario de la recepción de mensajes por red.

Bibliografía consultada

- Documentación sobre sockets en C.

- Material del curso relacionado con comunicación cliente-servidor.
- Referencias sobre entrada/salida no bloqueante.

Problema 4: Conflictos entre raylib y bibliotecas de Windows

Durante la integración del cliente gráfico con raylib, se presentaron conflictos al incluir bibliotecas relacionadas con Windows y sockets. Algunos headers podían entrar en conflicto con las utilizadas por raylib, provocando errores de compilación o comportamientos inesperados.

Intentos de solución

Inicialmente se intentó incluir directamente archivos relacionados con red y Windows dentro del cliente gráfico. Sin embargo, esto generó conflictos con la compilación del programa gráfico.

También se intentó reorganizar los include, pero esto no resolvía completamente el problema.

Solución encontrada

La solución fue evitar incluir directamente ciertas bibliotecas problemáticas dentro del archivo gráfico principal. En su lugar, se declararon únicamente los prototipos de las funciones de red necesarias para el cliente gráfico, manteniendo la implementación de red separada en sus propios archivos.

Con esto se logró que el cliente gráfico pudiera usar raylib sin entrar en conflicto con las funciones de red.

Recomendación

Se recomienda mantener separadas las responsabilidades de cada archivo. El archivo gráfico debe encargarse principalmente de la interfaz y el dibujo, mientras que las funciones de red deben mantenerse en archivos separados. Esto reduce conflictos y facilita el mantenimiento del código.

Conclusión

Este problema permitió mejorar la organización del cliente C y separar la interfaz gráfica de la comunicación por sockets.

Bibliografía consultada

- Documentación oficial de raylib.
- Documentación de WinSock.
- Material de apoyo sobre cómo modular en C.

Problema 5: Mejora visual y estabilidad del cliente gráfico

Después de lograr compilar y ejecutar el cliente gráfico, se observó que era necesario mejorar la visualización de la pantalla. Algunos elementos podían quedar muy cerca de los bordes, el HUD podía verse saturado y era necesario diferenciar mejor jugadores, aliens, bunkers, proyectiles y ovni.

Intentos de solución

Primero se realizaron pruebas modificando manualmente posiciones, tamaños y textos en pantalla. Sin embargo, algunos cambios provocaban que ciertos elementos quedaran desalineados o que se dificultara ver correctamente la partida.

También se probaron ajustes en el tamaño de ventana y en el área de juego para mejorar la distribución visual.

Solución encontrada

Se ajustó el tamaño de la ventana gráfica, el área de juego, la posición del HUD y la forma de dibujar entidades. También se agregó diferenciación visual entre jugadores conectados. Además, se incorporaron elementos visuales para representar el estado de los bunkers, las vidas, los puntajes, la velocidad y el modo espectador.

Recomendación

Se recomienda evitar saturar el HUD con demasiada información en un espacio pequeño.

Conclusión

Este problema permitió mejorar la experiencia visual del juego y facilitar la demostración del proyecto durante la revisión.

Bibliografía consultada

- Documentación oficial de raylib.
- Referencias del juego original Space Invaders.
- Pruebas visuales realizadas durante el desarrollo.

Problema 6: Colisiones poco precisas entre disparos y entidades

Durante las pruebas del juego se detectó que algunos disparos debían impactar demasiado cerca del centro de los aliens para ser reconocidos como colisión. Esto hacía que el juego se sintiera poco natural, ya que visualmente parecía que un disparo tocaba a un enemigo, pero el servidor no lo detectaba como impacto.

También se revisó el comportamiento de los bunkers, ya que debían proteger correctamente al jugador de los disparos enemigos.

Intentos de solución

Primero se revisó si el problema estaba en el dibujo del cliente gráfico, pero se determinó que el cliente solamente representaba el estado enviado por el servidor. Por lo tanto, el problema estaba en la lógica de colisiones del servidor.

También se probaron ajustes visuales, pero esto no resolvía el problema, porque la colisión real se calculaba en Java.

Solución encontrada

Se ajustaron las dimensiones y la tolerancia de las zonas de impacto en el servidor Java. Se agregó un margen de colisión para que los impactos fueran más coherentes con lo que el jugador veía en pantalla.

Además, se corrigió el orden de revisión de colisiones con bunkers y jugadores para que los bunkers pudieran absorber disparos correctamente antes de que estos impactaran al jugador.

Recomendación

Se recomienda que en videojuegos con interfaz gráfica se pruebe la coherencia entre lo que se dibuja y lo que se calcula internamente. Si la zona visual de un objeto es más grande que su zona de colisión, el jugador puede percibir errores aunque el código esté funcionando según sus coordenadas internas.

Conclusión

Este problema permitió mejorar la jugabilidad y hacer que las colisiones fueran más justas para el usuario.

Bibliografía consultada

- Referencias sobre detección de colisiones en videojuegos 2D.
- Material visto en el curso relacionado con estructuras y lógica de programación.
- Pruebas realizadas directamente sobre el cliente gráfico.

Problema 7: Integración del control físico ESP32 por UART

El proyecto requería un control físico de dos botones. En la implementación del grupo, se utilizó una ESP32 con autorización del profesor. La ESP32 debía enviar comandos al cliente para mover el cañón y disparar.

El problema principal fue definir cómo integrar el control físico sin modificar la lógica central del servidor Java ni romper el funcionamiento del cliente gráfico con teclado.

Intentos de solución

Primero se analizó si la ESP32 debía comunicarse directamente con el servidor. Sin embargo, esta opción complicaba la arquitectura, ya que el servidor está diseñado para recibir acciones desde clientes jugadores.

También se consideró que la ESP32 actuara como una entrada del cliente, lo cual resultó más coherente con la arquitectura del proyecto.

Solución encontrada

Se implementó soporte UART en el cliente gráfico C. La ESP32 envía comandos simples por serial:

L: izquierda
R: derecha
F: disparar

El cliente C lee estos comandos desde el puerto COM y los convierte en acciones internas:

L: MOVE_LEFT
R: MOVE_RIGHT
F: FIRE

Luego, el cliente envía esas acciones al servidor por sockets, igual que cuando el jugador usa el teclado.

Recomendación

Se recomienda verificar antes de la defensa cuál es el puerto COM de la ESP32 y asegurarse de que ningún monitor serial esté usando ese puerto. Si el puerto está abierto en otra aplicación, el cliente no podrá acceder a él.

Conclusión

Esta solución permitió integrar el control físico sin alterar la lógica principal del servidor. La ESP32 funciona como dispositivo de entrada del cliente gráfico y el servidor continúa siendo el encargado de procesar las reglas del juego.

Bibliografía consultada

- Documentación de UART en ESP32.
- Documentación sobre comunicación serial en Windows.
- Material visto en algunos cursos relacionado con comunicación entre dispositivos.

Problema 8: Manejo del repositorio y archivos generados

Durante el desarrollo se generaron archivos de compilación como ejecutables, archivos .jar y carpetas target. Estos archivos no debían mezclarse con el código fuente principal del repositorio, ya que podían generar conflictos o aumentar innecesariamente el tamaño del proyecto.

Intentos de solución

Inicialmente se revisó el estado del repositorio con:

```
git status
```

Se observaron archivos generados por compilación que no era necesario versionar. También se identificó que usar git add . podía subir archivos no deseados.

Solución encontrada

Se configuró el repositorio para ignorar archivos generados y se decidió agregar únicamente los archivos fuente, scripts y documentos necesarios. También se evitó subir ejecutables generados cuando no eran necesarios como parte del código fuente.

Para guardar cambios importantes se utilizaron comandos específicos como:

```
git add archivo específico  
git commit -m "mensaje descriptivo"  
git push origin integración-final
```

Recomendación

Se recomienda revisar siempre git status antes de realizar un commit. También se recomienda evitar git add . cuando existen archivos generados dentro del proyecto.

Conclusión

Este problema permitió mantener el repositorio más limpio, organizado y adecuado para la revisión del profesor.

Bibliografía consultada

- Documentación oficial de Git.
- Documentación de GitHub.
- Consejos relacionados con bitácora y control de versiones.

Conclusión general de los problemas encontrados

Los problemas encontrados durante el desarrollo permitieron mejorar la calidad técnica del proyecto. La mayoría de dificultades estuvieron relacionadas con configuración del entorno, integración de librerías, comunicación entre componentes y ajustes de jugabilidad.

Cada problema fue analizado, probado y corregido de forma incremental. Como resultado, el proyecto cuenta con un servidor Java funcional, un cliente gráfico en C, comunicación por sockets, soporte para dos jugadores, espectador, comandos administrativos, control físico por ESP32 y una estructura organizada en el repositorio.

La experiencia permitió reforzar la importancia de probar continuamente, documentar los cambios realizados y mantener una arquitectura clara entre servidor, cliente gráfico y control físico.

7. Conclusiones del proyecto

El proyecto SpaCEinvaders permitió aplicar de forma práctica los conceptos de programación orientada a objetos en Java y programación imperativa en C, integrando ambos paradigmas dentro de una misma solución cliente-servidor. La implementación del servidor en Java y el cliente gráfico en C demostró la capacidad de combinar tecnologías diferentes para desarrollar una aplicación distribuida funcional.

Se logró cumplir con los principales requisitos establecidos para la tarea, incluyendo la comunicación mediante sockets, el soporte para múltiples jugadores, el modo espectador, la administración centralizada de la lógica del juego y la implementación de un control físico mediante comunicación UART. Además, se utilizaron estructuras de datos adecuadas para representar las entidades del juego y mantener sincronizado el estado de la partida entre todos los clientes conectados.

El desarrollo del proyecto permitió reforzar conocimientos relacionados con programación concurrente, comunicación en red, diseño de software orientado a objetos, manejo de estructuras de datos y desarrollo de interfaces gráficas. Asimismo, se comprobó la importancia de mantener una arquitectura bien definida donde cada componente tenga responsabilidades claramente separadas.

Otro aspecto importante fue la utilización de herramientas de control de versiones para coordinar el trabajo del equipo y mantener un historial organizado de los cambios realizados durante el desarrollo. Esto facilitó la integración de nuevas funcionalidades y la resolución de problemas encontrados durante las distintas etapas del proyecto.

En conclusión, spaCEinvaders constituye una implementación funcional que cumple los objetivos planteados en la asignación y permitió fortalecer habilidades técnicas relacionadas con el desarrollo de software, el trabajo colaborativo y la integración de componentes de hardware y software dentro de una misma aplicación.

8. Recomendaciones del proyecto

Como trabajo futuro, se recomienda ampliar las funcionalidades del juego incorporando nuevas mecánicas, diferentes tipos de enemigos y niveles con dificultad progresiva. Esto permitiría enriquecer la experiencia de juego y aprovechar la arquitectura cliente-servidor ya implementada.

Se recomienda continuar fortaleciendo la modularidad del código, manteniendo una clara separación entre la lógica del servidor, la interfaz gráfica del cliente y los componentes de comunicación. Esta práctica facilita el mantenimiento, la corrección de errores y la incorporación de nuevas funcionalidades.

Para futuras versiones, sería conveniente implementar mecanismos más avanzados de sincronización y manejo de concurrencia en el servidor, especialmente si se desea aumentar la cantidad de jugadores conectados simultáneamente o agregar nuevas interacciones dentro de la partida.

Respecto al control físico, se recomienda documentar claramente la configuración del puerto serial, las dependencias necesarias y el procedimiento de conexión del dispositivo antes de cada ejecución. Esto facilitará la instalación y utilización del sistema en diferentes equipos.

Finalmente, se recomienda mantener una documentación técnica actualizada y continuar utilizando herramientas de control de versiones como Git para registrar los cambios realizados en el proyecto, facilitar el trabajo colaborativo y mejorar la trazabilidad del desarrollo.

9. Bibliografía consultada

Apache Maven Project. (s. f.). *Maven getting started guide*. Apache Maven. Recuperado el 15 de junio de 2026, de <https://maven.apache.org/guides/getting-started/>

Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/en/v2>

Espressif Systems. (s. f.). *Universal asynchronous receiver/transmitter (UART)*. ESP-IDF Programming Guide. Recuperado el 15 de junio de 2026, de <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/uart.html>

Free Software Foundation. (s. f.). *GCC online documentation*. GNU Project. Recuperado el 15 de junio de 2026, de <https://gcc.gnu.org/onlinedocs/>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Microsoft. (2021, 12 de octubre). *socket function (winsock2.h)*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-socket>

Microsoft. (2023, 9 de febrero). *Getting started with Winsock*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/winsock/getting-started-with-winsock>

MSYS2. (s. f.). *Package management*. MSYS2 Documentation. Recuperado el 15 de junio de 2026, de <https://www.msys2.org/docs/package-management/>

Oracle. (s. f.). *Class ServerSocket*. Java Platform, Standard Edition 17 API Specification. Recuperado el 15 de junio de 2026, de <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/ServerSocket.html>

Oracle. (s. f.). *Class Socket*. Java Platform, Standard Edition 17 API Specification. Recuperado el 15 de junio de 2026, de <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/net/Socket.html>

raylib. (s. f.). *raylib cheatsheet*. Recuperado el 15 de junio de 2026, de <https://www.raylib.com/cheatsheet/cheatsheet.html>

Taito Corporation. (s. f.). *The evolution of Space Invaders*. Google Arts & Culture. Recuperado el 15 de junio de 2026, de https://artsandculture.google.com/story/the-evolution-of-space-invaders/wAVxCu_0BaX0aw